You said:

i am using list operations in python in rag i want to practice all the exmples

ChatGPT said:

Great! Since you are learning **RAG (Retrieval-Augmented Generation) using Python**, practicing Python list operations is important. Lists are used extensively in RAG pipelines for document chunks, embeddings, retrieval results, metadata, and context construction.

I recommend learning list operations through practical examples related to RAG rather than only theoretical examples.

Python List Operations – Complete Practice Guide

We'll cover 30+ examples across 8 categories.

1. Creating and accessing lists

Python

```python
documents = [
    "Python is a programming language",
    "RAG combines retrieval and generation",
    "LangChain helps build LLM applications",
    "Vector databases store embeddings"
]

# 1. Print entire list
print(documents)

# 2. Get first element
print(documents[0])

# 3. Get last element
print(documents[-1])

# 4. Get elements from index 1 to 3
print(documents[1:3])

# 5. Get first three elements
print(documents[:3])
```

2. Adding elements to lists

Python

```python
documents = ["Python", "LangChain"]

# 9. Append one element
documents.append("Vector DB")

# 10. Extend with multiple elements
documents.extend(["Embeddings", "LLM"])

# 11. Insert at specific index
documents.insert(1, "RAG")
```

```
print(documents)
```

Difference between append() and extend():

Python

```
a = ["Python", "Java"]

a.append(["C#", "C++"])
print(a)

# ['Python', 'Java', ['C#', 'C++']]

b = ["Python", "Java"]

b.extend(["C#", "C++"])
print(b)

# ['Python', 'Java', 'C#', 'C++']
```

3. Removing elements

Python

```
documents = [
    "Python",
    "Java",
    "C#",
    "JavaScript",
    "Python"
]

# 12. Remove specific value
documents.remove("Java")

# 13. Remove element using index
documents.pop(1)

# 14. Remove last element
documents.pop()

# 15. Delete using del
del documents[0]

# 16. Clear entire list
```

4. Searching and filtering

These operations are especially useful when retrieving documents in RAG.

Python

```
documents = [
    "Python programming",
    "Machine learning",
    "Python for AI",
    "Deep learning",
    "RAG applications"
]

# 17. Check whether an element exists
print("Python programming" in documents)
```

```python
# 18. Find index
print(documents.index("Machine learning"))

# 19. Count occurrences
print(documents.count("Python programming"))

# 20. Filter documents containing Python
python_docs = [
    doc for doc in documents
    if "Python" in doc
```

5. Sorting and reversing

Python

```python
scores = [0.85, 0.45, 0.95, 0.72, 0.65]

# 21. Sort ascending
scores.sort()

# 22. Sort descending
scores.sort(reverse=True)

# 23. Create sorted copy
new_scores = sorted(scores)

# 24. Reverse list
scores.reverse()

print(scores)
```

6. List comprehension

Very important for RAG document processing.

Python

```python
documents = [
    "Python is easy",
    "RAG is powerful",
    "LangChain is useful",
    "Vector database stores vectors"
]

# 25. Convert all documents to uppercase
upper_docs = [doc.upper() for doc in documents]

# 26. Get documents containing RAG
rag_docs = [doc for doc in documents if "RAG" in doc]

# 27. Get document lengths
doc_lengths = [len(doc) for doc in documents]

# 28. Remove empty documents
clean_docs = [doc for doc in documents if doc.strip()]

print(upper_docs)
print(rag_docs)
```

7. Nested lists – RAG document chunks

In RAG, a document can be split into multiple chunks.

Python

```python
document_chunks = [
    ["Python introduction", "Python variables"],
    ["RAG architecture", "RAG retrieval"],
    ["LangChain chains", "LangChain agents"]
]

# 29. Access nested list
print(document_chunks[0])

# 30. Access nested element
print(document_chunks[1][0])

# 31. Loop through nested list
for document in document_chunks:
    for chunk in document:
        print(chunk)

# 32. Flatten nested list
flat_chunks = [
    chunk
    for document in document_chunks
```

8. Combining lists – RAG retrieval results

Python

```python
query_results = ["Python basics", "Python functions"]

vector_results = ["Python basics", "Python OOP"]

# 33. Combine lists
all_results = query_results + vector_results

# 34. Remove duplicates
unique_results = list(set(all_results))

# 35. Zip two lists
documents = ["Doc1", "Doc2", "Doc3"]
scores = [0.95, 0.87, 0.76]

results = list(zip(documents, scores))

print(results)

# 36. Enumerate
for index, document in enumerate(documents):
    print(index, document)
```

---

9. Practical RAG mini project

Let's combine these list operations into a small document retrieval example.

Python

```python
documents = [
    "Python is a programming language",
    "Python supports machine learning",
    "RAG uses document retrieval",
    "RAG combines LLM with external knowledge",
    "LangChain supports RAG applications",
    "",
    "Vector databases store embeddings"
```

```
]

# Step 1: Remove empty documents
clean_documents = [
    doc for doc in documents if doc.strip()
]

# Step 2: Search documents using query
query = "RAG"

retrieved_documents = [
    doc for doc in clean_documents
```

This is a simplified keyword-based retrieval example. Real RAG systems generally use embeddings and similarity search.

---

10. Your Python list practice exercises

Try these yourself without looking at the solutions initially.

Practice checklist 0/8 completed

Beginner

1. Remove duplicate documents from a list.

Show hint

Beginner

2. Find all documents containing the word Python.

Show hint

Intermediate

3. Find the top 3 similarity scores from a list.

Show hint

Intermediate

4. Combine two retrieval result lists and remove duplicates while preserving order.

Show hint

Intermediate

5. Flatten a nested list of document chunks.

Show hint

Intermediate

6. Pair each document with its similarity score.

Show hint

Advanced

7. Retrieve documents whose similarity score is greater than 0.75.

Show hint

Advanced

8. Extract the top 3 documents based on similarity score.

Show hint

Recommended learning sequence for RAG

Priority	Python concept	RAG application
1	List operations	Document management
2	List comprehension	Document filtering
3	Nested lists	Document chunking
4	Dictionary	Metadata management
5	Tuple	Document-score pairs
6	Set	Duplicate removal
7	Zip and enumerate	Retrieval processing
8	Sorting	Top-K retrieval

My recommendation: Don't stop with lists. Practice dictionaries, sets, tuples, and list comprehension next because these are heavily used in LangChain, document loaders, retrievers, and RAG pipelines.

Which practice format would you prefer next?

1. **50 Python list coding problems** – beginner to advanced, with solutions.
2. **RAG-focused Python exercises** – practical document chunking, filtering, embeddings, and Top-K retrieval.

You said:

please provide much examples in enumerate

ChatGPT said:

Absolutely! Python's `enumerate()` is very useful in RAG applications because it allows you to iterate over a list while getting both the **index and the value**.

Let's practice 35+ examples, starting from beginner level and moving towards real-world RAG applications.

1. Understanding enumerate()

Syntax:

Python

```python
enumerate(iterable, start=0)
```

- `iterable` : List, tuple, string, etc.
- `start` : Starting index (default is 0).

Example 1: Basic enumerate

Python

```
documents = ["Python", "Java", "C#", "Angular"]

for index, value in enumerate(documents):
    print(index, value)
```

Output:

```
Plain text

0 Python
1 Java
2 C#
3 Angular
```

Example 2: Start index from 1

```
Python

documents = ["Python", "Java", "C#"]

for index, value in enumerate(documents, start=1):
    print(index, value)
```

Output:

```
Plain text

1 Python
2 Java
3 C#
```

Example 3: Store enumerate result in a list

```
Python

documents = ["Python", "Java", "C#"]

result = list(enumerate(documents))

print(result)
```

Output:

```
Python

[(0, 'Python'), (1, 'Java'), (2, 'C#')]
```

2. Basic enumerate operations

Example 4: Print index and value separately

Python

```python
languages = ["Python", "Java", "C#", "JavaScript"]

for index, language in enumerate(languages):
    print("Index:", index)
    print("Language:", language)
```

Example 5: Find a particular element and its index

Python

```python
documents = ["Python", "Java", "RAG", "LangChain"]

for index, doc in enumerate(documents):
    if doc == "RAG":
        print("Found at index:", index)
```

Example 6: Find elements containing Python

Python

```python
documents = [
    "Python programming",
    "Java programming",
    "Python machine learning",
    "RAG architecture"
]

for index, doc in enumerate(documents):
    if "Python" in doc:
        print(index, doc)
```

Example 7: Print only elements at even indexes

Python

```python
documents = ["Doc1", "Doc2", "Doc3", "Doc4", "Doc5"]

for index, doc in enumerate(documents):
    if index % 2 == 0:
        print(index, doc)
```

Output:

Plain text

```
0 Doc1
2 Doc3
4 Doc5
```

Example 8: Print elements at odd indexes

Python

```python
for index, doc in enumerate(documents):
    if index % 2 != 0:
        print(index, doc)
```

3. Enumerate with list comprehension

This is particularly important for data processing.

Example 9: Create index-value pairs

Python

```python
documents = ["Python", "RAG", "LangChain"]

result = [(index, doc) for index, doc in enumerate(documents)]

print(result)
```

Example 10: Filter and return indexes

Python

```python
documents = ["Python", "Java", "Python AI", "RAG"]

result = [
    (index, doc)
    for index, doc in enumerate(documents)
    if "Python" in doc
]

print(result)
```

Example 11: Add numbering to documents

Python

```python
documents = ["Introduction", "Architecture", "Implementation"]

numbered_docs = [
    f"Document {index + 1}: {doc}"
    for index, doc in enumerate(documents)
]

print(numbered_docs)
```

Example 12: Extract documents based on index

Python

```python
documents = ["A", "B", "C", "D", "E"]
```

```python
selected_docs = [
    doc for index, doc in enumerate(documents)
    if index >= 2
]

print(selected_docs)
```

4. Enumerate with conditions

Example 13: Find the first matching document

Python

```python
documents = ["Java", "C#", "Python", "RAG"]

for index, doc in enumerate(documents):
    if "Python" in doc:
        print("Found:", doc)
        print("Index:", index)
        break
```

Example 14: Find all matching indexes

Python

```python
documents = ["Python", "Java", "Python", "RAG", "Python"]

indexes = []

for index, doc in enumerate(documents):
    if doc == "Python":
        indexes.append(index)

print(indexes)
```

Output:

Plain text

```
[0, 2, 4]
```

Example 15: Update elements using enumerate

Python

```python
documents = ["python", "java", "csharp"]

for index, doc in enumerate(documents):
    documents[index] = doc.upper()

print(documents)
```

Output:

Plain text

```
['PYTHON', 'JAVA', 'CSHARP']
```

Example 16: Skip empty documents

Python

```python
documents = [
    "Python",
    "",
    "RAG",
    " ",
    "LangChain"
]

for index, doc in enumerate(documents):
    if not doc.strip():
        continue

    print(index, doc)
```

5. Enumerate with multiple data structures

`enumerate()` works with more than lists.

Example 17: Enumerate a tuple

Python

```python
languages = ("Python", "Java", "C#")

for index, language in enumerate(languages):
    print(index, language)
```

Example 18: Enumerate a string

Python

```python
text = "RAG"

for index, character in enumerate(text):
    print(index, character)
```

Example 19: Enumerate dictionary keys

Python

```python
student = {
    "name": "Krishna",
    "course": "GenAI",
    "language": "Python"
}

for index, key in enumerate(student):
```

```python
    print(index, key, student[key])
```

Example 20: Enumerate dictionary items

```python
student = {
    "name": "Krishna",
    "course": "GenAI",
    "language": "Python"
}

for index, (key, value) in enumerate(student.items()):
    print(index, key, value)
```

6. Enumerate with zip – Very important for RAG

Suppose you have documents and similarity scores from a vector database.

Example 21: Combine documents and scores

```python
documents = ["Python basics", "RAG architecture", "LangChain"]
scores = [0.95, 0.89, 0.78]

for index, (doc, score) in enumerate(zip(documents, scores)):
    print(index, doc, score)
```

Example 22: Filter documents by similarity score

```python
documents = ["Python basics", "RAG architecture", "LangChain"]
scores = [0.95, 0.65, 0.88]

for index, (doc, score) in enumerate(zip(documents, scores)):
    if score > 0.80:
        print(index, doc, score)
```

Example 23: Extract Top-K documents

```python
documents = ["Doc1", "Doc2", "Doc3", "Doc4"]
scores = [0.75, 0.98, 0.85, 0.92]

results = sorted(
    zip(documents, scores),
    key=lambda x: x[1],
    reverse=True
)

top_k = results[:2]

for index, (doc, score) in enumerate(top_k, start=1):
```

```
print(f"Rank {index}: {doc}, Score: {score}")
```

Output:

```
Plain text
Rank 1: Doc2, Score: 0.98
Rank 2: Doc4, Score: 0.92
```

7. Enumerate with nested lists

Example 24: Enumerate nested document chunks

```python
documents = [
    ["Python basics", "Python functions"],
    ["RAG architecture", "RAG retrieval"],
    ["LangChain", "Vector DB"]
]

for doc_index, chunks in enumerate(documents):
    print("Document:", doc_index)

    for chunk_index, chunk in enumerate(chunks):
        print("  Chunk:", chunk_index, chunk)
```

Example 25: Enumerate flattened chunks

```python
chunks = [
    ["Chunk1", "Chunk2"],
    ["Chunk3", "Chunk4"]
]

for index, chunk in enumerate(
    [item for sublist in chunks for item in sublist]
):
    print(index, chunk)
```

Example 26: Enumerate with starting index for nested lists

```python
documents = [
    ["Python", "Java"],
    ["RAG", "LangChain"]
]

for doc_index, chunks in enumerate(documents, start=1):
    for chunk_index, chunk in enumerate(chunks, start=1):
        print(f"Document {doc_index}, Chunk {chunk_index}: {chunk}")
```

8. Real-world RAG applications

These examples connect enumerate to the RAG workflow.

Example 27: Number document chunks

Python

```python
text = """
Python is a programming language.
RAG is a retrieval augmented generation technique.
LangChain helps build LLM applications.
Vector databases store embeddings.
"""

chunks = text.strip().split("\n")

for index, chunk in enumerate(chunks, start=1):
    print(f"Chunk ID: {index}")
    print(f"Content: {chunk}")
```

Example 28: Create metadata for chunks

Python

```python
chunks = [
    "Python is easy",
    "RAG uses retrieval",
    "LangChain supports LLMs"
]

chunk_metadata = []

for index, chunk in enumerate(chunks):
    metadata = {
        "chunk_id": index,
        "content": chunk,
        "length": len(chunk)
    }

    chunk_metadata.append(metadata)

print(chunk_metadata)
```

Example 29: Create document objects

Python

```python
documents = [
    "Python programming",
    "Machine learning",
    "RAG architecture"
]

rag_documents = []

for index, content in enumerate(documents):
    rag_documents.append({
        "id": f"doc_{index}",
        "page_content": content,
        "metadata": {
            "source": "training_data",
            "index": index
        }
```

```
    })

for doc in rag_documents:
    print(doc)
```

Example 30: Simulate RAG retrieval results

```python
retrieved_docs = [
    "Python supports list operations",
    "Python supports dictionary operations",
    "RAG uses document retrieval",
    "LangChain supports retrievers"
]

scores = [0.96, 0.89, 0.92, 0.81]

for rank, (doc, score) in enumerate(
    zip(retrieved_docs, scores), start=1
):
    print(f"Rank: {rank}")
    print(f"Document: {doc}")
    print(f"Similarity: {score}")
    print("----------------")
```

---

9. Advanced enumerate examples

Example 31: Find the maximum score and its index

```python
scores = [0.72, 0.95, 0.88, 0.99, 0.65]

max_index, max_score = max(
    enumerate(scores),
    key=lambda x: x[1]
)

print(max_index, max_score)
```

Example 32: Find the minimum score and its index

```python
scores = [0.72, 0.95, 0.88, 0.99, 0.65]

min_index, min_score = min(
    enumerate(scores),
    key=lambda x: x[1]
)

print(min_index, min_score)
```

Example 33: Enumerate and create a dictionary

Python

```python
documents = ["Python", "Java", "RAG"]

doc_dict = {
    index: doc
    for index, doc in enumerate(documents)
}

print(doc_dict)
```

Output:

Python

```
{0: 'Python', 1: 'Java', 2: 'RAG'}
```

Example 34: Find duplicate elements with indexes

Python

```python
documents = ["Python", "Java", "Python", "RAG", "Java"]

seen = {}

for index, doc in enumerate(documents):
    if doc in seen:
        print(
            f"Duplicate: {doc}, "
            f"First index: {seen[doc]}, "
            f"Current index: {index}"
        )
    else:
        seen[doc] = index
```

Example 35: Enumerate with multiple conditions

Python

```python
scores = [0.45, 0.90, 0.78, 0.95, 0.60]

for index, score in enumerate(scores, start=1):
    if score >= 0.80:
        print(f"Result {index}: Relevant ({score})")
    else:
        print(f"Result {index}: Not relevant ({score})")
```

10. Practice exercises

Try these without looking at the previous examples.

Enumerate coding challenge 0/8 solved

Beginner

1. Print every element in a list along with its index starting from 1.

Write your Python solution here...

Show hint

Beginner

2. Find the index of every document containing Python.

Write your Python solution here...

Show hint

Intermediate

3. Update all negative numbers in a list to zero using enumerate.

Write your Python solution here...

Show hint

Intermediate

4. Find the highest similarity score and its index.

Write your Python solution here...

Show hint

Intermediate

5. Combine document chunks with their IDs using enumerate.

Write your Python solution here...

Show hint

Advanced

6. Use enumerate and zip to display documents with similarity scores and ranks.

Write your Python solution here...

Show hint

Advanced

7. Find duplicate documents and print both their original and duplicate indexes.

Write your Python solution here...

Show hint

Advanced

8. Create a nested dictionary containing document IDs, chunks, and metadata using enumerate.

Write your Python solution here...

Show hint

Important: Understand enumerate vs range

Operation	Purpose
`for x in list`	Gets values only
`for i in range(len(list))`	Gets indexes
`for i, x in enumerate(list)`	Gets both index and value
`enumerate(list, start=1)`	Gets both, starting at 1

My suggestion: Focus especially on Examples 21–34. These are useful for your RAG learning, particularly when working with document chunks, retrieval scores, metadata, and Top-K results.

You can also send me your solutions to the 8 exercises, and I'll review your Python code like a coding interview.

You said:

please provide examples for range anlso

ChatGPT said:

Absolutely! Python's `range()` is another important concept, especially when working with lists, document chunks, loops, indexing, and RAG pipelines.

Let's learn `range()` from beginner to advanced with 40 practical examples.

1. Understanding Python range()

The syntax is:

Python

```
range(start, stop, step)
```

Parameter	Description
`start`	Starting number (default 0)
`stop`	Ending limit (excluded)
`step`	Increment or decrement (default 1)

Example 1: Basic range

Python

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

Remember: `range(5)` generates numbers from 0 to 4, not 5.

Example 2: Specify start and stop

```python
for i in range(1, 6):
    print(i)
```

Output:

```
1
2
3
4
5
```

Example 3: Specify step

```python
for i in range(0, 11, 2):
    print(i)
```

Output:

```
0
2
4
6
8
10
```

Example 4: Negative step

```python
```

```
for i in range(10, 0, -1):
    print(i)
```

Output:

```
10
9
8
7
6
5
4
3
2
1
```

Example 5: Convert range into list

```
numbers = list(range(1, 11))

print(numbers)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

2. Range with lists

This is where `range()` becomes very useful.

Example 6: Iterate through a list using indexes

```
languages = ["Python", "Java", "C#", "Angular"]

for i in range(len(languages)):
    print(i, languages[i])
```

Example 7: Access only list values

```python
documents = ["Doc1", "Doc2", "Doc3"]

for i in range(len(documents)):
    print(documents[i])
```

Example 8: Access list elements in reverse order

Python

```python
documents = ["Doc1", "Doc2", "Doc3", "Doc4"]

for i in range(len(documents) - 1, -1, -1):
    print(documents[i])
```

Example 9: Update list elements using range

Python

```python
numbers = [10, 20, 30, 40]

for i in range(len(numbers)):
    numbers[i] = numbers[i] * 2

print(numbers)
```

Output:

Python

```
[20, 40, 60, 80]
```

Example 10: Update only even-indexed elements

Python

```python
numbers = [10, 20, 30, 40, 50, 60]

for i in range(0, len(numbers), 2):
    numbers[i] = 0

print(numbers)
```

Output:

Python

```
[0, 20, 0, 40, 0, 60]
```

3. Range with conditions

Example 11: Print even numbers

Python

```python
for i in range(1, 21):
    if i % 2 == 0:
        print(i)
```

Example 12: Print odd numbers

Python

```python
for i in range(1, 21):
    if i % 2 != 0:
        print(i)
```

Example 13: Find numbers divisible by 5

Python

```python
for i in range(1, 51):
    if i % 5 == 0:
        print(i)
```

Example 14: Sum numbers from 1 to 100

Python

```python
total = 0

for i in range(1, 101):
    total += i

print(total)
```

Example 15: Multiplication table

Python

```python
number = 5

for i in range(1, 11):
    print(f"{number} x {i} = {number * i}")
```

4. Range with list comprehension

Very useful for Python data processing.

Example 16: Create a list of numbers

```python
numbers = [i for i in range(10)]

print(numbers)
```

Example 17: Create even numbers

```python
even_numbers = [
    i for i in range(1, 21)
    if i % 2 == 0
]

print(even_numbers)
```

Example 18: Create squares

```python
squares = [i ** 2 for i in range(1, 11)]

print(squares)
```

Example 19: Create a list with indexes

```python
documents = ["Python", "Java", "RAG"]

result = [
    (i, documents[i])
    for i in range(len(documents))
]

print(result)
```

Output:

```python
[(0, 'Python'), (1, 'Java'), (2, 'RAG')]
```

Example 20: Filter elements using indexes

```python
documents = [
    "Python",
    "Java",
    "RAG",
    "LangChain",
    "Vector DB"
]

result = [
    documents[i]
    for i in range(len(documents))
    if "a" in documents[i].lower()
]

print(result)
```

5. Range with nested loops

Nested loops are useful when processing nested lists and document chunks.

Example 21: Basic nested range

```python
for i in range(3):
    for j in range(3):
        print(i, j)
```

Example 22: Create a multiplication table

```python
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

Example 23: Access nested lists

```python
documents = [
    ["Python", "Java"],
    ["RAG", "LangChain"],
    ["Azure", "OpenAI"]
]

for i in range(len(documents)):
    for j in range(len(documents[i])):
        print(documents[i][j])
```

Example 24: Update nested list elements

Python

```
numbers = [
    [1, 2],
    [3, 4],
    [5, 6]
]

for i in range(len(numbers)):
    for j in range(len(numbers[i])):
        numbers[i][j] *= 10

print(numbers)
```

Output:

Python

```
[[10, 20], [30, 40], [50, 60]]
```

6. Advanced range operations

Example 25: Find the maximum number using range

Python

```
numbers = [10, 50, 20, 90, 30]

maximum = numbers[0]

for i in range(1, len(numbers)):
    if numbers[i] > maximum:
        maximum = numbers[i]

print(maximum)
```

Example 26: Find the minimum number

Python

```
numbers = [10, 50, 20, 90, 30]

minimum = numbers[0]

for i in range(1, len(numbers)):
    if numbers[i] < minimum:
        minimum = numbers[i]

print(minimum)
```

Example 27: Find duplicate elements

Python

```python
numbers = [10, 20, 30, 20, 40, 10]

for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] == numbers[j]:
            print("Duplicate:", numbers[i])
```

Example 28: Compare adjacent elements

Python

```python
numbers = [10, 20, 15, 30, 25]

for i in range(len(numbers) - 1):
    if numbers[i] < numbers[i + 1]:
        print(numbers[i], "is smaller than", numbers[i + 1])
```

Example 29: Reverse a list without reverse()

Python

```python
numbers = [10, 20, 30, 40, 50]

reversed_list = []

for i in range(len(numbers) - 1, -1, -1):
    reversed_list.append(numbers[i])

print(reversed_list)
```

Example 30: Find the second-largest number

Python

```python
numbers = [10, 50, 20, 90, 30]

unique_numbers = list(set(numbers))
unique_numbers.sort(reverse=True)

print(unique_numbers[1])
```

7. Range in RAG applications

Now let's apply `range()` to practical RAG scenarios.

Example 31: Split documents into chunks

Python

```python
document = """
```

```
Python is a programming language.
RAG is a retrieval technique.
LangChain is a framework.
Vector databases store embeddings.
"""

sentences = document.strip().split("\n")

chunk_size = 2

for i in range(0, len(sentences), chunk_size):
    chunk = sentences[i:i + chunk_size]

    print(f"Chunk {i // chunk_size + 1}")
    print(chunk)
```

Output:

Plain text

```
Chunk 1
['Python is a programming language.', 'RAG is a retrieval technique.']

Chunk 2
['LangChain is a framework.', 'Vector databases store embeddings.']
```

Example 32: Create chunk IDs

Python

```
chunks = [
    "Python introduction",
    "Python functions",
    "RAG architecture",
    "Vector database"
]

for i in range(len(chunks)):
    print({
        "chunk_id": f"chunk_{i}",
        "content": chunks[i]
    })
```

Example 33: Process Top-K retrieval results

Python

```
documents = ["Doc1", "Doc2", "Doc3", "Doc4", "Doc5"]

scores = [0.75, 0.95, 0.85, 0.90, 0.60]

top_k = 3

sorted_indexes = sorted(
    range(len(scores)),
    key=lambda i: scores[i],
    reverse=True
)

for rank in range(top_k):
    index = sorted_indexes[rank]
```

```
    print(
        rank + 1,
        documents[index],
        scores[index]
    )
```

Example 34: Batch processing documents

Suppose you have 100 documents and want to process 10 documents at a time.

Python

```python
documents = [f"Document {i}" for i in range(100)]

batch_size = 10

for i in range(0, len(documents), batch_size):

    batch = documents[i:i + batch_size]

    print(f"Processing batch {i // batch_size + 1}")
    print(batch)
```

Example 35: Generate metadata for document chunks

Python

```python
chunks = [
    "Python basics",
    "Python functions",
    "RAG architecture"
]

metadata = []

for i in range(len(chunks)):

    metadata.append({
        "chunk_id": i,
        "source": "python_guide.pdf",
        "content": chunks[i],
        "length": len(chunks[i])
    })

print(metadata)
```

Example 36: Simulate embedding batch processing

Python

```python
documents = [
    "Python basics",
    "RAG architecture",
    "LangChain",
    "Vector database",
    "LLM applications"
]

batch_size = 2

for i in range(0, len(documents), batch_size):
```

```
    batch = documents[i:i + batch_size]

    print("Sending documents for embedding:")
    print(batch)
```

This simulates batching; it does not generate actual embeddings.

8. Difference between range(), enumerate(), and list iteration

This is very important for interviews.

Consider:

Python

```
documents = ["Python", "Java", "RAG"]
```

Method	Example	What you get
Direct iteration	`for x in documents`	Values
Range	`for i in range(len(documents))`	Index
Enumerate	`for i, x in enumerate(documents)`	Index and value
Range with step	`range(0, 10, 2)`	Numbers with increments

Compare:

Python

```
# 1. Direct iteration
for doc in documents:
    print(doc)

# 2. Range
for i in range(len(documents)):
    print(i, documents[i])

# 3. Enumerate
for i, doc in enumerate(documents):
    print(i, doc)
```

When should you use which?

- Use direct iteration when you only need values.
- Use `range()` when you need index-based access, custom steps, or to modify list elements.
- Use `enumerate()` when you need both index and value.

---

9. Your range() coding exercises

Try these 10 problems. They progress from beginner to advanced.

Range practice tracker 0/10 completed

Beginner

1. Print numbers from 1 to 50 using range().

Write your Python code here...

Show hint

Beginner

2. Print numbers from 100 to 1 in reverse order.

Write your Python code here...

Show hint

Beginner

3. Print all even numbers between 1 and 100.

Write your Python code here...

Show hint

Beginner

4. Find the sum of all elements in a list using range().

Write your Python code here...

Show hint

Intermediate

5. Reverse a list without using reverse() or slicing.

Write your Python code here...

Show hint

Intermediate

6. Find duplicate elements using nested range loops.

Write your Python code here...

Show hint

Intermediate

7. Split 100 documents into batches of 5 using range().

Write your Python code here...

Show hint

Advanced

8. Find the top 3 documents based on similarity scores.

Write your Python code here...

Show hint

Advanced

9. Create chunks from a list of 20 sentences, with each chunk containing 4 sentences.

Write your Python code here...

Show hint

Advanced

10. Compare every document with the next document and find consecutive duplicate documents.

Write your Python code here...

Show hint

My recommendation for your RAG learning

Practice these concepts in this order:

1. `range()` – index-based operations.
2. `enumerate()` – index and value together.
3. `zip()` – combining documents and scores.
4. `list comprehension` – filtering and transforming documents.
5. Nested loops – processing document chunks.
6. `sorted()` and `lambda` – ranking retrieval results.

Once you understand these six concepts, you will be much more comfortable writing Python code for document loading, chunking, embedding batches, and retrieval pipelines.